

Chapitre 2 : Base de données

Rappels sur les bases de données

Les bases de données (BDD) permettent de stocker des données. Mais concrètement, comment cela fonctionne-t-il ? En quelques mots, il s'agit d'un système de fichiers contenant les données de votre application. Cependant, ces fichiers sont totalement transparents pour l'utilisateur d'une base de données, donc totalement transparents pour vous ! La différence avec les fichiers classiques se trouve dans le fait que ce n'est pas vous qui les gérez : c'est votre BDD qui les organise, les range et, le cas échéant, vous retourne les informations qui y sont stockées. De plus, plusieurs utilisateurs peuvent accéder simultanément aux données dont ils ont besoin, sans compter que de nos jours, les applications sont amenées à traiter une grande quantité de données, le tout en réseau. Imaginez-vous gérer tout cela manuellement alors que les BDD le font automatiquement...

Pour utiliser une BDD, vous avez besoin de deux éléments : la base de données et ce qu'on appelle le SGBD (Système de Gestion de Base de Données).

Définitions :

JDBC est une API (Application Programming Interface) Java disponible depuis la version 1.1 du JDK. Cette API est constituée d'un ensemble d'interfaces et de classes qui permettent l'accès, à partir de programmes Java, à des données tabulaires (c.-à-d. triées sous forme de table ou de tableur). Par données tabulaires, on entend généralement des bases de données contenues dans des SGBD relationnels. Mais, JDBC n'est pas restreinte à ce type de source de données. On peut aussi accéder à des sources de données sous forme de fichiers (fichiers XML par exemple).

Rappelons d'abord quelques notions élémentaires.

SQL (Structured Query Language) : le « langage de requêtes structuré » est un langage d'interrogation universel permettant la définition, la manipulation et le contrôle des données, pour les bases de données relationnelles. SQL est un standard supporté par la plupart des SGBD.

SGBDR (Système de gestion de bases de données relationnelles) : un SGBDR est un logiciel assurant la structuration, le stockage, la maintenance, la mise à jour et la consultation d'une base de données relationnelle. Sachant qu'une base de données relationnelle est une collection de données structurées sous forme de tableaux à deux dimensions (lignes et colonnes). Exemples : Oracle, Sybase, MySQL, DB2, Interbase, etc.

ODBC (Open DataBase Connectivity) : interface écrite en C qui permet la communication avec les bases de données. Tout comme JDBC, ODBC permet la manipulation des bases de données « indépendamment » du SGBD.

L'API JDBC permet, entre autres :

- l'établissement d'une connexion avec le SGBD ;
- l'envoi de requêtes SQL au SGBD, à partir du programme Java ;
- le traitement, au niveau du programme, des données retournées par le SGBD ;
- le traitement des métadonnées de la connexion, de l'instruction ou des résultats ;
- le traitement des erreurs retournées par le SGBD lors de l'exécution d'une instruction.

Voici une liste non exhaustive des SGBD accessibles avec JDBC :

- DB2
- FoxPro
- Ingres
- Interbase
- MS Access
- MS SQL Server
- MySQL
- Oracle
- Paradox
- PostgreSQL
- Sybase

Utilisez JDBC (Java DataBase Connectivity)

Les étapes généralement rencontrées dans un programme utilisant cette technologie :

- chargement du driver correspondant au SGBD ciblé ;
- ouverture d'une connexion vers la base de données ;
- création d'une requête (Statement) ;
- exécution de la requête ;
- accès aux résultats de la requête ;
- accès aux métadonnées (des résultats, de la connexion, etc.) ;
- libération des ressources (fermeture de la connexion, etc.).

1. Chargement Driver

Un pilote ou driver JDBC est un « logiciel » qui permet d'établir une connexion entre un programme Java et un système de gestion de bases de données. Ce « logiciel » est en fait une implémentation de l'interface Driver, du package java.sql.

La première chose à faire pour pouvoir se connecter à une base MySql est d'ajouter une librairie permettant de le faire facilement : [mysql-connector-java/6.0.6](https://dev.mysql.com/doc/connector-java/6.0.6/en/mysql-connector-java-6.0.6.html).

Utiliser la méthode Class.forName, qui aura pour effet d'enregistrer le Driver auprès du DriverManager.

```
String nomDriver = "nom_du_driver";
try{
    Class.forName(nomDriver);
}catch(ClassNotFoundException cnfe){
    System.out.println("La classe "+nomDriver+" n'a pas été trouvée");
    cnfe.printStackTrace();
}
```

Voici quelques exemples :

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");  
//pour le pont JDBC-ODBC  
Class.forName("com.mysql.jdbc.Driver");  
//pour MySQL et ConnectorJ
```

2. Connexion vers la base de données :

Une connexion à une base de données avec JDBC est représentée par une instance de la classe `java.sql.Connection`. Pour ouvrir une connexion vers une base de données, il suffit de spécifier le chemin de connexion, le nom d'utilisateur et le mot de passe, à la méthode `getConnection` de `DriverManager`.

```
String URL = "URL";  
String login = "log";  
String password = "pass";  
try{  
    Connection connection = DriverManager.getConnection(URL,login,password);  
    //interaction avec la base  
}  
catch(SQLException sqle){  
    //cf. Comment gérer les erreurs ?  
}  
finally{  
    //cf. Comment bien fermer une connexion ?  
}
```

Les URL JDBC sont définies sous forme de String selon ce schéma :

```
String url = "jdbc:<subprotocol>:<subname>"
```

Exemples d'url:

- jdbc:odbc:maBase → Access
- jdbc:mysql://localhost/maBase → MySQL
- jdbc:oracle:oci8@:maBase → Oracle
- jdbc:oracle:thin:@localhost:8000:maBase → Oracle
- jdbc:sybase:Tds:localhost:5020/maBase → SyBase

3. Création et exécution d'une requête :

L'interface Statement :

L'interface `Statement` représente une instruction SQL. L'obtention d'une instance de cette interface se fait à partir de la `Connection` :

```
try{  
    Connection connection = ...  
    Statement statement = connection.createStatement();  
}  
catch(Exception e){  
    //cf. Comment gérer les erreurs ?  
}
```

Une deuxième méthode permet de spécifier le type des ResultSet créés avec ce Statement.

```
Statement createStatement(int resultSetType,  
                           int resultSetConcurrency,  
                           int resultSetHoldability)
```

Voici la liste des différents arguments acceptés :

1. **Pour les possibilités de déplacement** (int resultSetType) :

ResultSet.TYPE_FORWARD_ONLY : c'est la valeur par défaut (et la seule possible pour JDBC 1.0). Elle indique que les déplacements du curseur ne peuvent se faire « qu'en avant » (de la première à la dernière ligne). Une fois la dernière ligne atteinte (la méthode last renvoyant true) le ResultSet est fermé et les données ne sont plus accessibles ;

ResultSet.TYPE_SCROLL_INSENSITIVE : cette valeur indique que le curseur peut être déplacé dans les deux sens, mais aussi arbitrairement (de manière absolue ou relative). Le terme insensitive indique que le ResultSet est insensible aux modifications des valeurs dans la base de données. Cela définit en fait une vue statique des données contenues dans le ResultSet ;

ResultSet.TYPE_SCROLL_SENSITIVE : cette valeur indique que le curseur peut être déplacé dans les deux sens, mais aussi arbitrairement (de manière absolue ou relative). Le terme sensitive indique que le ResultSet est sensible aux modifications des valeurs dans la base de données. Cela définit en fait une vue dynamique des données contenues dans le ResultSet.

2. **Pour les possibilités de mise à jour** (int resultSetConcurrency) :

ResultSet.CONCUR_READ_ONLY : c'est la valeur par défaut (et la seule possible pour JDBC 1.0). Elle indique que les données contenues dans le ResultSet ne peuvent qu'être lues ;

ResultSet.CONCUR_UPDATABLE : cette valeur indique que l'on peut modifier les données de la base via le ResultSet.

3. **Pour le maintien des curseurs** (int resultSetHoldability) :

ResultSet.HOLD_CURSORS_OVER_COMMIT : les objets ResultSet ne sont pas fermés. Ils restent ouverts lorsqu'une validation est effectuée implicitement ou explicitement ;

ResultSet.CLOSE_CURSORS_AT_COMMIT : les objets ResultSet sont fermés lorsqu'une validation est effectuée implicitement ou explicitement.

L'interface Statement propose une dizaine de méthodes permettant l'exécution d'expressions SQL. On peut les regrouper par catégories :

- ❖ les méthodes execute : méthodes génériques pour n'importe quelle expression SQL, elle renvoie une valeur booléenne.
- ❖ les méthodes executeQuery : méthodes appropriées à l'exécution d'un SELECT, elle renvoie un ResultSet.
- ❖ les méthodes executeUpdate : méthodes appropriées à l'exécution d'une instruction menant à la modification de la BD (INSERT, UPDATE, DELETE, CREATE, etc.), elle renvoie un entier.

Exemples :

```
Statement statement = connection.createStatement();  
boolean result = statement.execute("SELECT * FROM MATABLE");  
ResultSet resultSet = statement.executeQuery("SELECT ATTRIBUT1, ATTRIBUT2 FROM MA  
TABLE");  
int col = statement.executeUpdate("INSERT INTO MATABLE VALUES(15, 'bonjour', 7.0)");
```

L'interface PreparedStatement:

L'interface PreparedStatement étend Statement et représente une instruction paramétrée. Cette interface diffère de Statement sur deux points principaux :

- ❖ les instances de PreparedStatement contiennent une instruction SQL déjà compilée. D'où le terme prepared. Cela améliore notamment les performances si cette instruction doit être appelée de nombreuses fois ;
- ❖ les instructions SQL des instances de PreparedStatement contiennent un ou plusieurs paramètres d'entrée, non spécifiés lors de la création de l'instruction. Ces paramètres sont représentés par des points d'interrogation (?). Ces paramètres doivent être spécifiés avant l'exécution.

Le passage des paramètres d'entrée des PreparedStatement se fait grâce à l'ensemble des méthodes setXXX. Il est important de connaître les correspondances entre les types SQL et les types java

Exemples :

```
String sql = "UPDATE Stocks SET prix = ?, quantite = ? WHERE nom = ?";  
//préparation de la requête  
PreparedStatement preparedStatement = connection.prepareStatement(sql);  
//on assigne un décimal au premier paramètre  
preparedStatement.setBigDecimal(1,15.6);  
//on assigne un entier au second paramètre  
preparedStatement.setInt(2,256);  
//on assigne une chaîne de caractères au troisième  
preparedStatement.setString(3,"café");  
//exécution de la requête  
preparedStatement.executeUpdate();
```

ResultSet :

Les instances de l'interface ResultSet contiennent les résultats d'une requête SQL. Ils contiennent les tuples (lignes) satisfaisant aux conditions de la requête.

La structure des ResultSet est très semblable à celle d'une Table dans une base de données relationnelle :

- ❖ **les colonnes (column)** : ce sont les éléments verticaux de la table (ou ResultSet). Ils symbolisent les attributs des différents enregistrements de la table. Ils sont caractérisés par un nom et un domaine dans lequel ils puisent leur valeur (par exemple pour les entiers INTEGER ou VARCHAR pour les chaînes de caractères).
- ❖ **les lignes (row)** : ce sont les éléments horizontaux de la table. On les nomme aussi tuples ou n-uplets. Ils sont les différents enregistrements contenus dans la table (ou ResultSet). Chaque ligne renseigne les attributs définis par les colonnes.
- ❖ **le curseur (cursor)** : cet objet pointe sur une « ligne ». Cette ligne peut être soit une ligne comme défini ci-dessus (contenant les données), soit une ligne spéciale comme afterLast, insertRow, etc. C'est le curseur qui permet le déplacement dans le ResultSet, pour l'accès aux différents enregistrements.

Voici le code Java permettant de récupérer toutes les valeurs contenues dans celle-ci :

```
Statement statement = connection.createStatement(
ResultSet.TYPE_FORWARD_ONLY,
ResultSet.CONCUR_READ_ONLY);
String sql = "SELECT * FROM MaTable";
ResultSet resultat = statement.executeQuery(sql);
while(resultat.next()){
int id = resultat.getInt(1);
String nom = resultat.getString(2);
double prix = resultat.getDouble(3);
java.sql.Date date = resultat.getDate(4);
int row = resultat.getRow();
System.out.println("Données contenues dans la ligne "+row);
System.out.println("id : "+id+" nom : "+nom+
" prix : "+prix+" date : "+date);
}
```

Exemple :

```
Statement statement = connection.createStatement(
ResultSet.TYPE_SCROLL_SENSITIVE,
ResultSet.CONCUR_READ_ONLY);
String sql = "SELECT * FROM MaTable";
ResultSet resultat = statement.executeQuery(sql);
resultat.first();
//on récupère le "prix" de la première ligne
```

N.B. Naturellement cette fonctionnalité n'est disponible que pour les ResultSet vérifiant ResultSet.TYPE_SCROLL_SENSITIVE.

Parcours d'un ResultSet :

L'interface ResultSet fournit plusieurs méthodes pour parcourir les données. Ces différentes méthodes sont bien sûr disponibles selon le type de ResultSet. Notons qu'au contraire de la plupart des structures de données en Java (tableaux, vector, List, etc.) les index des lignes commencent à 1.

- **next** : passe le curseur à l'élément suivant ;
- **previous** : passe le curseur à l'élément précédent ;
- **first** : passe le curseur sur le premier élément ;
- **last** : passe le curseur sur le dernier élément ;
- **beforeFirst** : passe le curseur avant le premier élément (position par défaut du curseur) ;
- **afterLast** : passe le curseur après le dernier élément.

Les Métadonnées :

Les métadonnées sont les informations, accessibles via JDBC, sur une base de données ou sur le résultat d'une requête SQL. Ce sont en fait les informations accessibles en dehors des données elles-mêmes.

ResultSetMetaData : informations sur les types et propriétés des colonnes d'un ResultSet. Récupérable avec la méthode getMetaData de ResultSet ou de PreparedStatement.

Pour connaître le nombre de colonnes d'un `ResultSet`, il suffit d'utiliser `ResultSetMetaData`. Par exemple :

```
String sql = "SELECT * FROM MATABLE";
Statement statement = connection.createStatement();
ResultSet resultat = statement.executeQuery(sql);
ResultSetMetaData metadata = resultat.getMetaData();
int nombreColonnes = metadata.getColumnCount();
System.out.println("Ce ResultSet contient "+nombreColonnes+" colonnes.");
```

Par contre, il n'existe pas de méthode toute faite pour récupérer le nombre de lignes d'un `ResultSet`. Cette fonctionnalité a été retirée des premières spécifications, car les implémentations étaient trop gourmandes en ressources. Il existe plusieurs possibilités pour connaître cette valeur.

Si le `ResultSet` est de type `TYPE_SCROLL_XXX`, on peut implémenter le code suivant :

```
String sql = "SELECT * FROM MATABLE";
Statement statement = connection.createStatement();
ResultSet resultat = statement.executeQuery(sql);
//on place le curseur sur le dernier tuple
resultat.last();
//on récupère le numéro de la ligne
int nombreLignes = resultat.getRow();
//on replace le curseur avant la première ligne
resultat.beforeFirst();
System.out.println("Ce ResultSet contient "+nombreLignes+" lignes.");
```

L'interface **`ResultSetMetaData`** permet de récupérer les noms des colonnes d'un `ResultSet`. Deux méthodes sont à notre disposition :

- **`getColumnName(int column)`** : `String` : Cette méthode renvoie le nom de la colonne dont l'index est *column*.
- **`getColumnLabel(int column)`** : `String` : Cette méthode renvoie le nom, suggéré pour l'affichage, de la colonne dont l'index est *column*

Trois méthodes de **`ResultSetMetaData`** peuvent nous renseigner sur le type SQL et sur la classe Java correspondante d'une colonne donnée.

- **`getColumnClassName(int index)`** : renvoie une chaîne de caractères correspondant au nom complet de la classe Java de la colonne ciblée par *index*.
- **`getColumnType(int index)`** : renvoie un entier correspondant au type SQL de la colonne ciblée par *index*. Cet entier est un de ceux définis dans `java.sql.Types`.
- **`getColumnTypeName(int index)`** : renvoie une chaîne de caractères correspondant au type SQL de la colonne ciblée par *index*. Cette chaîne est le nom de la variable renvoyée par `getColumnType` dans `java.sql.Types`.

4. Déconnexion de la base :

La connexion est fermée automatiquement par le garbage collector, mais il est toujours préférable de fermer soi-même une connexion. De manière générale, il est nécessaire de bien gérer les exceptions.

```
if(connection!=null)
{
try{    connection.close();    }catch(Exception e){
    e.printStackTrace();
}
}
```